

COMP2611

Artificial Intelligence

Assignment 1: Search Algorithms

Muhammad-Abdullah Kashif-Khan (**Submitter**) - sc22makk@leeds.ac.uk

Mohammed Mahfuz Miah - sc22mm@leeds.ac.uk

Declaration:

We confirm that we have worked as pair on this project and both of us have made significant contributions to both parts of the assignment.

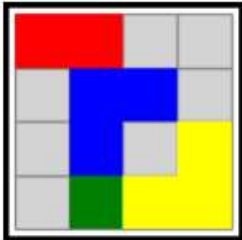
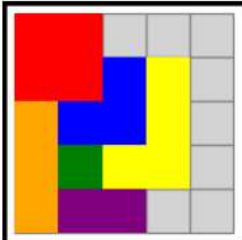
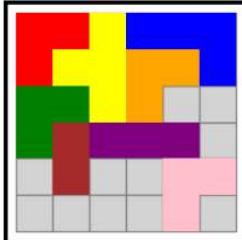
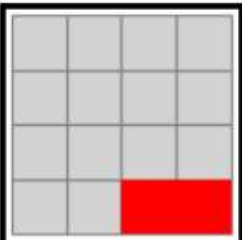
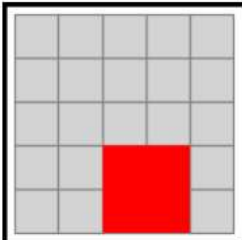
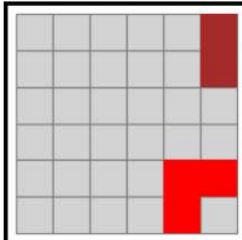
We are aware that both members of the pair will receive the same grade.

The submitter confirms that they have agreed the final submitted version of this report with the other member of the pair.

Part A: Sliding Blocks Puzzle Search Investigation

A1.a Puzzle test cases

For part A, we have designed the following cases for the Sliding Block Puzzle:

	Easy_ps (easy problem set)	Hard_ps	Very_hard_ps
Initial state			
Goal state			

When designing these test cases, we considered what factors affected the difficulty of the puzzle. One factor included the amount of packing and locking (fitting blocks in patterns to reduce the number of empty squares between clusters of shapes) non-goal blocks had to make. This increased complexity as these non-goal blocks would have to find a way to unpack themselves before the goal blocks can proceed. Another factor included the size of the puzzle to provide the search algorithms with a greater variety of possible options, and therefore wider search space. An additional factor included the number of goal blocks, where a higher number of these would (typically) increase the complexity of the search space, by requiring additional shuffles to reach the goal state. It is careful to note the complexity requirement of picking possible goal states, as picking many possible end states could actually result in the puzzle becoming easier to solve.

A1.b Heuristics

Each heuristic is adapted to any puzzle of varying grid size, number of goal blocks, blocks shapes and sizes. For the Sliding Blocks Puzzle, we designed the following 3 heuristics:

Manhattan heuristic:

```
def manhattan_heuristic(state):
    # Calling upon globalised goal state
    global goal_state

    # Obtain 1st square (typically top left most square in block) and store in dict alongside the respective colour
    goal_positions = {}
    colours = state.colours
    for colour_no in range(1, len(colours)):
        goal_position = goal_state.blockcells(colour_no)
        if len(goal_state.blockcells(colour_no)) != 0:
            goal_positions[colour_no] = goal_position[0]

    distance = 0
    # Access the dict to pick out colours and calculate distance between goal and final state
    for goal_colour, goal_colour_position in goal_positions.items():
        block_position = state.blockcells(goal_colour)
        first_square_in_block_position = block_position[0]
        distance += abs(goal_colour_position[0] - first_square_in_block_position[0]) + abs(goal_colour_position[1] - first_square_in_block_position[1])
    return distance
```

This heuristic calculates the Manhattan distance between the current position of the goal block, and the goal block's final state. The goal block will tend to prefer moving towards the final goal state rather than aimlessly drifting to all possible states.

Scared Block heuristic:

```
# Non goal blocks will tend to move away from the goal positions and blocks
# Ideally provides a clear path for a block to travel through
def scared_block_heuristic(state):
    # Calling upon globalised goal state
    global goal_state

    # Calculating max distance between 2 corners in a search space.
    max_moves = goal_state.ncols + goal_state.nrows - 2

    # Obtain 1st square (typically top left most square in block) of end goal and current block and store in dict alongside the respective colour
    goal_positions = {}
    block_positions = {}
    colours = state.colours
    for colour_no in range(1, len(colours)):
        # Check if colour is indeed a part of the goal state
        goal_position = goal_state.blockcells(colour_no)
        if len(goal_position) != 0:
            goal_positions[colour_no] = goal_position[0]
        # Find the current position of the goal block and store
        current_block_position = state.blockcells(colour_no)
        block_positions[colour_no] = current_block_position[0]

    weight = 0
    goal_distance = 0
    block_distance = 0
    # Access the dict to pick out colours and calculate distance between non-goal blocks vs goal blocks and final state
    for goal_colour, goal_colour_position in goal_positions.items():
        block_position = block_positions.get(goal_colour)
        # Loop over non-goal blocks
        for colour_no in range(1, len(colours)):
            non_goal_block = state.blockcells(colour_no)
            # Check if colour is in the puzzle and colour is indeed a non-goal colour
            if colour_no != goal_colour and len(non_goal_block) != 0:
                # Calculate distances and weights
                first_square_in_non_goal_block_position = non_goal_block[0]
                goal_distance += abs(goal_colour_position[0] - first_square_in_non_goal_block_position[0]) + abs(goal_colour_position[1] - first_square_in_non_goal_block_position[1])
                block_distance += abs(block_position[0] - first_square_in_non_goal_block_position[0]) + abs(block_position[1] - first_square_in_non_goal_block_position[1])
                weight = max_moves - (goal_distance + block_distance)
    return weight
```

This heuristic makes non-goal blocks “run away” from the goal blocks and final goal block positions. This is dependent on the proximity between the blocks, where the closer the block is, the stronger the tendency to move away. This ideally helps to accelerate the unpacking of packed or locked clusters of blocks, and helps to create direct routes between the goal block and the final goal block position

Combination heuristic:

```
def combination_heuristic(state):
    return manhattan_heuristic(state) + scared_block_heuristic(state)
```

This heuristic combines the best of both heuristics mentioned above, encouraging goal blocks to move towards the final state, and non-goal blocks to “clear a path” for the goal blocks.

A1.c Search Algorithm Test Sequence

After experimenting with various search options, we decided to cover the following tests,

- Full scale tests of Easy_ps, Hard_ps and Very_hard_ps, covering all heuristics in best first and A* search algorithms. This will be executed 3 times due to variation of the times (possibly due to network delays), where an average will be taken. Depth first random will be based on the last recorded data stored during the final iteration of the loop, since it carries random compute time that cannot be averaged.
- A full scale test to the effects of toggling loop checking on Easy_ps, but with reduced (100,000) maximum nodes (ran out of memory and crashed at 250,000 nodes). This will be executed once to determine whether the search succeeded or not.
 - The plan was to originally use a harder problem set, but every related search failed

We have decided on these tests since we believe these datasets will provide us with adequate information to answer a wide variety of questions.

```
def run_general_search(problem_set, loop_check = True, max_nodes = 250000, test_count = 3):
    global goal_state
    goal_state = problem_set.get_goal_state()

    BF_time = 0
    DF_time = 0
    Best_manhattan_time = 0
    Best_scared_block_time = 0
    Best_combination_time = 0
    A_star_manhattan_time = 0
    A_star_scared_block_time = 0
    A_star_combination_time = 0

    for iteration in range(test_count):
        # Breadth first
        BF = search(problem_set, 'BF/FIFO', max_nodes, loop_check=loop_check, randomise=False, show_state_path=False, return_info=True, dots=False, show_path=False)
        BF_time = BF_time + float(BF["search_stats"]["time_taken"])

        # Depth first
        DF = search(problem_set, 'DF/LIFO', max_nodes, loop_check=loop_check, randomise=False, show_state_path=False, return_info=True, dots=False, show_path=False)
        DF_time = DF_time + float(DF["search_stats"]["time_taken"])

        # Depth first with random ordering (NOT taking an average time, since its random)
        DF_random = search(problem_set, 'DF/LIFO', max_nodes, loop_check=loop_check, randomise=True, show_state_path=False, return_info=True, dots=False, show_path=False)

        # Best first using manhattan heuristic
        Best_manhattan = search(problem_set, 'BF/FIFO', max_nodes, heuristic=manhattan_heuristic,
                                loop_check=loop_check, randomise=False, show_state_path=False, return_info=True, dots=False, show_path=False)
        Best_manhattan_time = Best_manhattan_time + float(Best_manhattan["search_stats"]["time_taken"])

        # Best first using scared blocks heuristic
        Best_scared_block = search(problem_set, 'BF/FIFO', max_nodes, heuristic=scared_block_heuristic,
                                    loop_check=loop_check, randomise=False, show_state_path=False, return_info=True, dots=False, show_path=False)
        Best_scared_block_time = Best_scared_block_time + float(Best_scared_block["search_stats"]["time_taken"])

        # Best first using combination heuristic
        Best_combination = search(problem_set, 'BF/FIFO', max_nodes, heuristic=combination_heuristic,
                                    loop_check=loop_check, randomise=False, show_state_path=False, return_info=True, dots=False, show_path=False)
        Best_combination_time = Best_combination_time + float(Best_combination["search_stats"]["time_taken"])

        # A* using manhattan heuristic
        A_star_manhattan = search(problem_set, 'BF/FIFO', max_nodes, heuristic=manhattan_heuristic, cost=cost,
                                    loop_check=loop_check, randomise=False, show_state_path=False, return_info=True, dots=False, show_path=False)
        A_star_manhattan_time = A_star_manhattan_time + float(A_star_manhattan["search_stats"]["time_taken"])

        # A* using scared blocks heuristic
        A_star_scared_block = search(problem_set, 'BF/FIFO', max_nodes, heuristic=scared_block_heuristic, cost=cost,
                                        loop_check=loop_check, randomise=False, show_state_path=False, return_info=True, dots=False, show_path=False)
        A_star_scared_block_time = A_star_scared_block_time + float(A_star_scared_block["search_stats"]["time_taken"])

        # A* using combination of both
        A_star_combination = search(problem_set, 'BF/FIFO', max_nodes, heuristic=combination_heuristic, cost=cost,
                                        loop_check=loop_check, randomise=False, show_state_path=False, return_info=True, dots=False, show_path=False)
        A_star_combination_time = A_star_combination_time + float(A_star_combination["search_stats"]["time_taken"])

    BF["search_stats"]["time_taken"] = float(BF_time / float(test_count))
    DF["search_stats"]["time_taken"] = float(DF_time / float(test_count))
    Best_manhattan["search_stats"]["time_taken"] = float(Best_manhattan_time / float(test_count))
    Best_scared_block["search_stats"]["time_taken"] = float(Best_scared_block_time / float(test_count))
    Best_combination["search_stats"]["time_taken"] = float(Best_combination_time / float(test_count))
    A_star_manhattan["search_stats"]["time_taken"] = float(A_star_manhattan_time / float(test_count))
    A_star_scared_block["search_stats"]["time_taken"] = float(A_star_scared_block_time / float(test_count))
    A_star_combination["search_stats"]["time_taken"] = float(A_star_combination_time / float(test_count))

    return (BF, DF, DF_random, Best_manhattan, Best_scared_block, Best_combination, A_star_manhattan, A_star_scared_block, A_star_combination)
```

Examples of calls to execute these tests are as follows:

```
easy_data = run_general_search(problem_set=easy_ps)
hard_data = run_general_search(problem_set=hard_ps)
very_hard_data = run_general_search(problem_set=very_hard_ps)
easy_data_loop_false = run_general_search(problem_set=easy_ps, loop_check=False, max_nodes=100000, test_count=1)
```

A2 Results

The following are the results from the tests conducted above (# = Number of).

Easy_ps results:

Search Algorithm	# Nodes set	Results	Path Length	# Nodes Generated	# Distinct Nodes	Time taken (s)
Breadth-first	250000	Success	10	3429	837	0.31
Depth-first	250000	Success	104	606	330	0.16
Depth-first, Random	250000	Success	131	705	387	0.19
Best-first, Manhattan Heuristic	250000	Success	11	157	80	0.15
Best-first, Scared Blocks Heuristic	250000	Success	11	365	181	0.17
Best-first, Combination Heuristic	250000	Success	11	209	99	0.18
A*, Manhattan Heuristic	250000	Success	10	1700	487	0.25
A*, Scared Blocks Heuristic	250000	Success	13	1714	572	0.3
A*, Combination Heuristic	250000	Success	14	853	335	0.21

Hard_ps results:

Search Algorithm	# Nodes set	Results	Path Length	# Nodes Generated	# Distinct Nodes	Time taken (s)
Breadth-first	250000	Success	29	47927	9565	3.3
Depth-first	250000	Success	1279	7905	3958	0.96
Depth-first, Random	250000	Success	1728	10497	5594	1.49
Best-first, Manhattan Heuristic	250000	Success	29	18511	3757	1.75
Best-first, Scared Blocks Heuristic	250000	Success	50	15031	4288	1.51
Best-first, Combination Heuristic	250000	Success	48	13644	3831	1.64
A*, Manhattan Heuristic	250000	Success	29	44362	8858	3.59
A*, Scared Blocks Heuristic	250000	Success	33	46372	9544	5.12
A*, Combination Heuristic	250000	Success	31	42653	8688	4.14

Very_hard_ps results:

Search Algorithm	# Nodes set	Results	Path Length	# Nodes Generated	# Distinct Nodes	Time taken (s)
Breadth-first	250000	Failed		1180709	250001	---
Depth-first	250000	Success	2212	15903	7692	2.05
Depth-first, Random	250000	Success	2499	18656	8829	2.14
Best-first, Manhattan Heuristic	250000	Success	88	5448	1963	1.16
Best-first, Scared Blocks Heuristic	250000	Success	75	10849	3320	1.86
Best-first, Combination Heuristic	250000	Success	75	7241	2500	1.42
A*, Manhattan Heuristic	250000	Success	54	639650	145172	87.21
A*, Scared Blocks Heuristic	250000	Success	65	361429	99400	59.47
A*, Combination Heuristic	250000	Success	66	215891	61018	41.5

Easy_ps (loop checking off) results:

Search Algorithm	# Nodes set	Results	Path Length	# Nodes Generated	Time taken (s)
Breadth-first	100000	Failed		100001	---
Depth-first	100000	Failed		100001	---
Depth-first, Random	100000	Success	142	763	0.15
Best-first, Manhattan Heuristic	100000	Success	11	2437	0.73
Best-first, Scared Blocks Heuristic	100000	Failed		100001	---
Best-first, Combination Heuristic	100000	Failed		100001	---
A*, Manhattan Heuristic	100000	Failed		100001	---
A*, Scared Blocks Heuristic	100000	Failed		100001	---
A*, Combination Heuristic	100000	Success	14	74573	15.19

A3 Observations

There are many observations that can be derived from the test data shown in [A2]. Some key, critical, and fascinating points observed concerning the performance of the search algorithms are as follows:

- Breadth-first search tended to produce the best path, but its lack in optimisation prevented it from performing optimally.
- Depth-first search quickly returned viable results, however the depth-first nature of the algorithm often returned suboptimal paths with (on occasion) obnoxiously long path lengths.
- Depth-first search with randomness is fast but very luck based. It relies on RNG to approach a path with relative optimality close or equal to the best path (which is often probabilistically rare). Overall, it is still subject to the same weaknesses as depth first search.
- Best first, to our surprise, produced some of the fastest (average time of 1 to 2 seconds on very_hard_ps), cheapest (in terms of nodes generated, therefore memory) and relatively good results. There is some nuance to the path lengths being much longer than something produced by for example A*, but if lowering the cost of time is the priority, best-first is best.
- A* successfully finds the middle ground between time cost and performance, promising results closest to the best path while taking a moderate yet reasonable amount of time to compute (approximately 1 min or less on very_hard_ps).

Concerning the performance of the heuristics:

- The Manhattan heuristic performed admirably, successfully outputting results very close in length to the best path, particularly in the A* search algorithm.
- The scared blocks heuristic performed reasonably well during execution. Although the path was longer than the Manhattan heuristic, it produced results much faster and with reduced memory consumed.
- It could perhaps be suggested that the synergy of the combination heuristic was a bit optimistic, as it often yielded suboptimal results. While the heuristic provided great response times with less memory consumption compared to the isolated heuristics, the path length was not optimal, which was not the expectation.
 - A possible hypothesis for this is that combination heuristics may operate better in different scenarios, where block splitting is much easier compared to the packed/locked cluster concept designed specifically to increase the difficulty of these puzzles.
 - Greater path lengths may be down to blocks moving away when not necessary, needlessly adding to the search solution.

Looking at investigating the effect of the loop checking algorithms, we see:

- A highly significant decrease in performance, increase in computation, memory use and time taken as the search space often tends to fail, even on the easy problem set.
- Crashes when running the program on too many (250,000) nodes, since the search algorithm consumes too much RAM.
- A rare comparison against those searches that have ended in success. Comparing the succeeded best first and A* algorithms reveal distinct node generated counts 2 orders of magnitude greater than their loop checking counterparts, while costing significantly more time to compute (in the case of A*).
- An overall essential need for loop checking algorithms when executing search algorithms.

Observing trends across all the problem sets, we can conclude:

- The smaller the problem set, the less a specific search algorithm matters since there is very little difference between the time taken, memory consumed, and final path generated.
- The larger the problem, the more profound the performance gap between worst operating algorithms (breadth-first and depth-first) and best search algorithms (Best and A*). This is due to the larger search space and greater locking as more complex shapes are introduced.

Part B: Robot Worker Scenario

B1 Robot worker scenario description

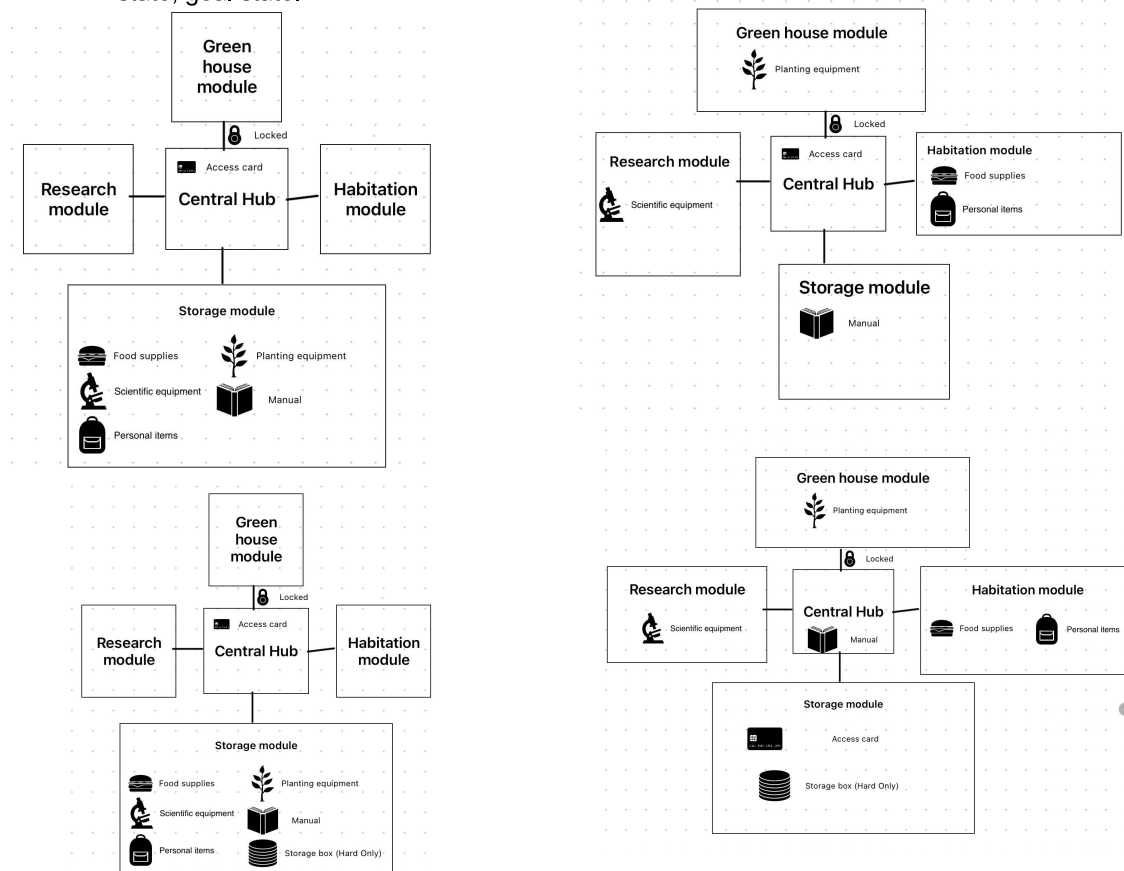
The robot is tasked with optimising resource distribution across a space station, comprising four specialised modules connected by a central hub. The modules include a Greenhouse, Habitation, Research, and Storage. Initially, all resources are stored in the Storage module, and the robot must efficiently allocate them to their designated locations, navigating through the central hub.

Key features:

- The Central hub's layout and the interconnections between modules mean the robot must constantly evaluate the most efficient paths and sequences of tasks.
- The Greenhouse module is initially locked, requiring the robot to retrieve an access card from the central hub to enter. This introduces a layer of complexity, as the robot must prioritise tasks to unlock this module.
- Items to be transported include Food Supplies (3 units), Scientific Equipment (6 units), Personal Items (2 units), a Manual (1 unit), Planting Equipment (4 units), and the Access Card (0 units). The weights add a constraint on the robot's carrying capacity, requiring strategic planning for movement as the robot will have a maximum strength of 15 units.
- The goal is to distribute the Planting Equipment to the Greenhouse (post-unlocking), Food Supplies and Personal Items to the Habitation module, and Scientific Equipment to the Research module. An additional challenge in the harder scenario involves relocating the Manual to the central hub and the Access Card back to the Storage module, further testing the robot's task management capabilities.

In the development of our scenario, a battery constraint was initially considered to mimic real-world challenges and test energy management in algorithms. Yet, changing battery levels would complicate state recognition, since loop checking analyses all variables of a state, including the different battery levels. We decided to exclude the battery feature to keep the scenario solvable and straightforward for algorithms, avoiding complexity from variable energy dynamics.

Below are the diagrams for our scenarios: From left to right: Easy start state, goal state, Hard start state, goal state.



B2 Heuristics

For our project we developed 2 heuristics for the robot scenario. A fairly simple one and a much more complex one.

The simple heuristic provides a basic measure of how far the robot is from completing its task by counting the number of room transitions needed to reach all target rooms from the robot's current location. This heuristic assumes each target room not currently occupied by the robot adds one to the distance count, offering a rudimentary estimate of the steps remaining to achieve the goal, without considering the specific tasks or item weights involved.

The complex heuristic combines three factors: distance to target rooms, weights of undelivered items, and actions needed for item handling. Distance is calculated similarly to the simple heuristic but adds weight factors for undelivered items, emphasising heavier items might require more energy to move (need to be carried with fewer items). The action factor considers picking up or dropping items, indicating the effort beyond just moving as this will increase path length. The action cost function assigns costs to actions (moving, picking up, putting down), with movement having a fixed cost and item handling varying based on item weight, reflecting the physical effort or energy consumption of these actions.

B3 Results

These are the generated test results for each scenario, complete with all search algorithms and their respective heuristics.

Easy scenario test results:

Search Method	# Node Set	Result	Path Length	# Nodes Generated	# Distinct Nodes	Time Taken (s)
Breadth-First	1000000	Success	16	1328929	344079	252.48
Depth-First	1000000	Success	11872	64801	43881	68.6
Depth-First Random	1000000	Success	24355	115922	81425	134.17
Best-first Simple heuristic	1000000	Success	16	930410	231130	169.22
Best-first Complex heuristic	1000000	Success	24	12418	4071	2.06
A* Simple heuristic	1000000	Success	16	1224257	321153	296.33
A* Complex heuristic	1000000	Success	16	51979	21207	11.63

Hard scenario test results:

Search Method	# Node Set	Result	Path Length	# Nodes Generated	# Distinct Nodes	Time Taken (s)
Breadth-First	1000000	Failed	N/A	3730868	1000001	736.14
Depth-First	100000	Failed	N/A	133903	100001	175.22
Depth-First Random	100000	Failed	N/A	129799	100001	206.05
Best-first Simple heuristic	1000000	Failed	N/A	3779545	1000001	835.68
Best-first Complex heuristic	1000000	Success	29	741	552	0.15
A* Simple heuristic	1000000	Failed	N/A	3805097	1000001	235.9
A* Complex heuristic	1000000	Success	22	891904	306629	214.6

B4 Key findings

Some general observations that can be derived from the test results and statistics are as follows:

- **Effectiveness of Heuristics:** The performance of heuristic-based searches (Best-first with simple and complex heuristics, A* with simple and complex heuristics) significantly varies between easy and hard problems, illustrating how the right heuristic can drastically reduce search time and complexity.
- **Scalability Issues:** Breadth-First Search (BFS) and Depth-First Search (DFS), along with their variations, show a marked difference in performance as the problem's complexity increases, indicating scalability issues with non-heuristic methods in more complex scenarios.

Other key specific observations related to each search problem as detailed below:

- **For our easy search space, we observed:**
 - **A High Success Rate:** All algorithms successfully solved the easy problem, but the efficiency varied greatly.
 - **Best-first with Complex Heuristic's Efficiency:** It was exceptionally efficient, solving the problem with the least number of nodes generated and in the shortest time (2.06s), highlighting the impact of a well-designed heuristic.
 - **A* with Complex Heuristic's Balanced Performance:** Exhibited a good balance between path length, nodes generated, and time taken, suggesting that A* with a complex heuristic is versatile and efficient across different metrics.
 - **Breadth-First's High Resource Consumption:** Despite success, BFS generated a large number of nodes and took significantly longer, indicating inefficiency in resource use for even relatively simpler problems.
- **For our hard search space, key observations were:**
 - **Challenges for Non-Heuristic Searches:** BFS, DFS, and its random variant failed to solve the hard problem within the node set limit, showcasing their limitations in tackling complex scenarios without heuristic guidance.
 - **Best-first with Complex Heuristic's Remarkable Efficiency:** It not only succeeded where others failed but also did so with an exceedingly low number of generated nodes and in an extremely short time (0.15s), underscoring the effectiveness of a powerful heuristic in overcoming complexity.
 - **A* with Complex Heuristic's Robustness:** Although taking longer and generating more nodes than in easy problems, it successfully solved the hard problem, proving its robustness and reliability in complex situations.

Finally, some conclusions we can derive from this investigation are:

- **Heuristics Make a Difference:** The data strongly indicates that the choice and design of heuristic can vastly improve search efficiency, particularly in complex scenarios.
- **Need for Balance Between Time and Resources:** While A* with the complex heuristic offered a balanced approach, it's evident that finding the right algorithm and heuristic combination is crucial for optimising both time and computational resources.
- **Complex Problems Require Advanced Strategies:** Basic search algorithms without heuristics struggle with complexity, highlighting the necessity for more sophisticated approaches for challenging tasks.